

Transaction → Task done in database

* Read [Retrival / See]

* Write [Insert / Update / Delete]

A	50
B	70

1. read_item(A)

2. $A := A - 50$

3. write_item(A)

4. read_item(B)

5. $B := B + 50$

6. write_item(B)

Transaction Property: → Ensured by: 1. Recovery
2. Concurrency control

*

A

Atomicity

[Recovery subsystem]

All or nothing

C

Consistency

[Both]

I

Isolation

[Concurrency control]

Prevents doing more than one transaction at a time

D

Durability

[Recovery subsystem]

Stays no matter what

(1) Atomicity:

1. read_item(A)

2. $A := A - 50$

3. write_item(A)

4. read_item(B)

5. $B := B + 50$

6. write_item(B)

(2) Consistency:

1. read_item(A)

2. $A := A - 50$

3. write_item(A)

4. read_item(B)

5. $B := B + 50$

6. write_item(B)

(3) Isolation:

T1	T2
read(A) $A := A - 50$ write(A)	
	read(A), read(B) print(A+B)
read(B) $B := B + 50$ write(B)	

(4) Durability: Last long

Question 8:

You are using your credit card to pay for an item online. You received an OTP on your phone which you entered incorrectly and pressed submit.

- ✓a. In this scenario, will the transaction execute completely or will it fail? Identify which ACID property ensures that the purchase does not take place and explain your answer.
- ✓b. If it fails, identify and explain if the DBMS will use the recovery system or the concurrency control mechanism.

Ans: (a) Atomicity
(b) Recovery subsystem

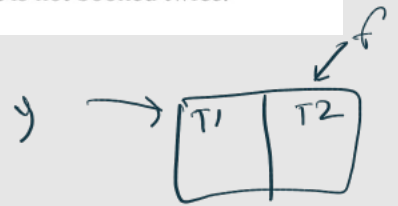
- a. An online shopping platform allows customers to place orders for products. Due to a software bug, a customer manages to place an order for a product that is out of stock. The system does not enforce the constraint to check product availability before accepting orders.

Ans: Consistency

Question 1:

Suppose you and your friend are both buying a train ticket to Sylhet from your own phones. There is one seat left. Both of you try to book that one seat, the database should not allow it, otherwise two people will have booked the same seat. Identify and explain which ACID property is necessary to ensure that the same seat is not booked twice.

Ans: Isolation
[Concurrency control]



Question 9:

Suppose you are a student at BRAC university. For the upcoming semester you have added 3 courses and paid your fees. Just before the start of the semester, the BRAC U server caught fire and all data was lost, this has created a huge problem for the university and many students, as many of you are not being able to add the same courses or same sections, or even the same number of courses even though most students have paid for their courses.

- a. Identify and explain which ACID property is necessary to ensure the above situation does not arise
- b. Identify and explain if the recovery system or the concurrency control mechanism will be required to ensure the property in (a).

Ans: (a) Durability
(b) Recovery



Recovery

- * To combat various failure: hardware, system crash
- * Even deadlock caused by concurrency control

Log based Recovery: * UNDO / Roll Back $\langle T_i, x, v \rangle$
 * ↳ History * Redo

$\langle T_i \text{ start} \rangle$

$\langle T_i, x, v1, v2 \rangle$
 No of transaction variable old value new value
 $\langle T_i \text{ end} \rangle$

$\langle T_0 \text{ start} \rangle$
 $\langle T_0, A, 1000, 950 \rangle$
 $\langle T_0, B, 2000, 2050 \rangle$
 $\langle T_0 \text{ commit} \rangle$

 $\langle T_1 \text{ start} \rangle$
 $\langle T_1, C, 700, 600 \rangle$
 $\langle T_1 \text{ commit} \rangle$

$\langle T_1 \text{ start} \rangle$
 $\langle T_1, A, 100, 150 \rangle$
 crash →
 $\langle T_1 \text{ commit} \rangle$

$\hat{T}_1$ Undo/Roll back

$\langle T_1 \text{ start} \rangle$
 $\langle T_1, A, 100, 150 \rangle$
 $\langle T_1 \text{ commit} \rangle$
 $\langle T_2 \text{ start} \rangle$
 $\langle T_2, B, 60, 70 \rangle$
 crash →
 $\langle T_2 \text{ commit} \rangle$

$\hat{T}_1$ Redo

T_2 Undo

Question 2:

For the given Log on the right, answer the following:

- Identify which transactions need to be undone and which need to be redone.
- Show the final values of A, B, and C after recovery.
- Explain why UNDO or REDO is required in this case.

Log :

```
<T1 START>
<T1, A, 100, 150>
<T1, B, 200, 250>
<T2 START>
<T2, C, 300, 350>
-- SYSTEM CRASH --
```

(a) Undo (b) A=100, B=200, C=300 (c)

Question 3:

For the given Log on the right, answer the following:

- Identify which transactions need to be undone and which need to be redone.
- Show the final values of A, B, and C after recovery.

Log :

```
<T1 START>
<T1, A, 500, 600>
<T1 COMMIT>
<T2 START>
<T2, B, 1000, 1200>
<T3 START>
<T3, C, 700, 800>
<T3 COMMIT>
-- SYSTEM CRASH --
```

(a) A, C → Redo
B → Undo

(b) A=600, B=1000, C=800

Question 7:

You are an employee at a travel agency. You added 5 tickets for a customer on a particular flight. After clicking 'pay' the tickets could not be bought as there were only 4 seats left instead of 5. In this scenario, identify if the DBMS will use the recovery system or the concurrency control mechanism. Explain your answer.

Ans: Recovery system



Concurrency Control

1. Serial Transaction → Takes more time

2. Parallel Transaction → Disk pressure

Problems: 1. Lost update

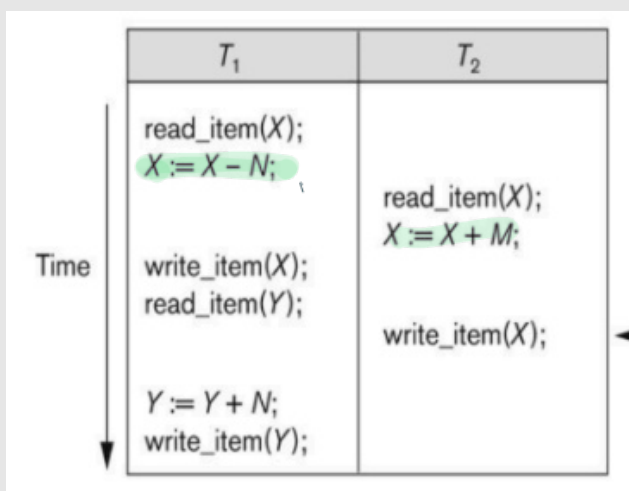
2. Temporary update / Dirty Read

3. Incorrect summary / Analysis Problem

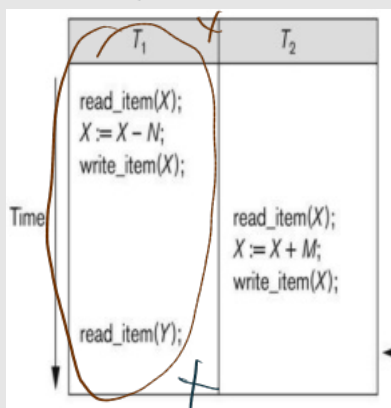
4. Non-repeatable read

5. Phantom Read

1. Lost Update: Overwrite



2. Temporary update / Dirty read: read uncommitted



3. Incorrect Summary Problem: Aggregate function before updating

↳ SUM
AVG

4. Non repeatable read: Same transaction but different value

5. Phantom read: New row appears

Isolation Level :

*strict
*slow
*consistent ←

	Dirty Read	Non-repeatable Read	Phantom Read
Serialization	X	X	X
Repeatable Read	X	X	✓
Read Committed	X	✓	✓
Read Uncommitted	✓	✓	✓

*Fast
*Least consistent ←

Question 4:

Two transactions are shown below. Identify which concurrency problem- Lost update, Temporary Update or Incorrect Summary, will occur in the following transaction and explain your answer.

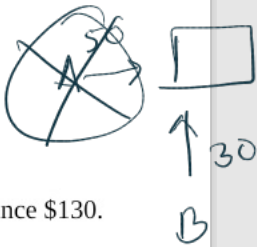
T1	T2
READ_ITEM (X) X=X+15	
	READ_ITEM (X) X=X-25 WRITE_ITEM (X)
WRITE_ITEM (X)	

Lost update

Question 10:

Imagine a banking application where two users, Alice and Bob, both have access to the same bank account. Let's say the current balance of the account is \$100.

- Alice checks the account balance and sees \$100.
- Bob also checks the account balance and sees \$100.
- Alice wants to transfer \$50 to her friend, so she initiates a transfer transaction.
- At the same time, Bob decides to deposit \$30 into the account.
- Now, Alice's transaction deducts \$50 from the account, so the balance becomes \$50.
- Before Alice's transaction is completed, Bob's transaction adds \$30 to the account, making the balance \$130.



Identify what type of concurrency problem the above scenario indicates and explain your answer.

Lost update

Question 5:

Two transactions A and B are shown below, Transaction A fails in the last step and rolls back. Identify which concurrency problem- Lost update, Temporary Update or Incorrect Summary, will occur in the following scenario and explain your answer.

A	B
Read_Item (X) X= X +100 Write (X)	
	Read_item (X) X = X + 500 Write_item (X)
Read_item (Y) Y = Y + X Write_item (Y) -- Transaction fails and rolls back --	

Dirty read/
Temporary
update problem

→ Aggregate

Question 6:

Two transactions A and B are shown below. In transaction A, the user wants to find the total of X and Y. Identify which concurrency problem- Lost update, Temporary Update or Incorrect Summary, will occur due to the following transactions and explain your answer.

A	B
Read_Item (X) Sum = 0 Sum = Sum + X	
	Read_item (Y) Y = Y + 500 Write_item (Y) Read_Item (X) X= X - 500 Write_item (X)
Read_item (Y) Sum = Sum + Y	

Ans. Incorrect summary

- T1: SELECT balance FROM account WHERE id = 1;
- T2: UPDATE account SET balance = 200 WHERE id = 1;
- T1: SELECT balance FROM account WHERE id = 1;

id	balance
1	100

Ans. Non repeatable read

- T1: SELECT * FROM orders WHERE amount > 100;
- T2: INSERT INTO orders (id, amount) VALUES (99, 150);
- T1: Repeats query row add

100	

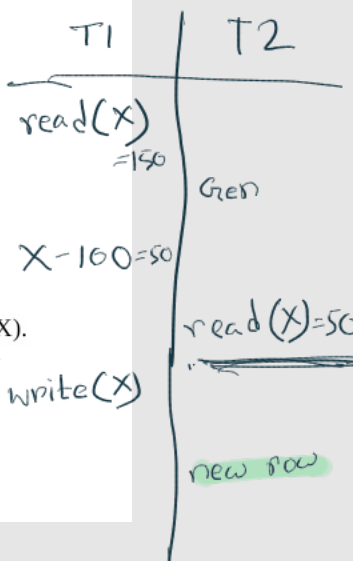
Ans. Phantom

Question 14

A banking database tracks customer transactions. Two transactions run concurrently:

Transaction T1 (Teller): Processes a withdrawal for Customer X.
Transaction T2 (Auditor): Generates a report on all customers with a balance below \$100.

- ✓ T1 reads Customer X's balance as \$150.
 - ✓ T2 starts generating a report and reads Customer X's balance as \$150.
 - ✓ T1 deducts \$100 from Customer X's balance, setting it to \$50 (but has **not committed yet**).
 - ✓ T2 reads Customer X's balance again and sees the **uncommitted \$50** value.
 - ✓ T1 **rollbacks** the transaction, reverting Customer X's balance to \$150.
 - ✓ Meanwhile, T2 **inserts a new customer**, Customer Y, with a balance of \$80.
 - ✓ T2 re-runs the query for customers with a balance below \$100 and sees Customer Y (but not Customer X).
- Identify the concurrency problems in the scenario. Provide your reason.
 - State which ACID property is being violated for each problem.
 - How can these issues be prevented?
 - What type of log based recovery needs to be used to revert back to the original balance?



(a) uncommitted → Dirty read
new row → Phantom read

(b) Isolation

(c) Concurrency control

(d) Undo

